
Three Agents Are Not Three Verifiers: Measuring Common-Mode Failure in LLM Verification

Adarsh Agrawal
Senior Member, IEEE

Abstract

Multiple language-model judgments are often treated as independent evidence that code or a proof is correct. We test that assumption using 1,152 judgments from six model families on executable code and proof artifacts, followed by a study of 141 candidates from 47 Python repositories. On the controlled held-out set, a cross-model trio fails together on 17.0% of candidates, 5.7 times the 3.0% rate obtained by multiplying its members' error rates. The excess appears in every evaluation domain and across three alternative model pairs. In the repository study, cross-model joint failure is 19.2%, while a six-model panel lowers it to 8.5% but remains five times its independence estimate. Model-only portfolios also trade lower wrong acceptance for much lower correct acceptance. A same-trigger policy that routes only model disagreements to pytest changes that tradeoff: on 130 labeled candidates it accepts 8.5% of wrong and 78.9% of correct changes, compared with 13.6% and 45.1% when the same disagreements are routed to another model. Finally, candidates optimized against three repeated judgments reach a 55.0% accepted-but-wrong rate. These results support a deployment rule: measure shared errors directly, and use model diversity to route uncertain cases to a qualitatively different check.

1 Introduction

Language models increasingly review generated code, patches, proofs, and specifications. A common reliability strategy is to ask several agents and combine their votes. Repeated calls appear to add redundancy, while different model families appear to add diversity. Neither choice guarantees independent evidence.

Voting helps only when voters are individually useful and their errors do not overlap too often [13, 14, 19]. Model verifiers can share training sources, conventions, prompt interpretations, and blind spots. Different explanations can therefore end in the same wrong decision.

We measure this common-mode risk on formal artifacts and repository changes. Our main statistic is *all-member failure*, the fraction of candidates for which every verifier in a panel is wrong. We compare the observed rate with the product of the members' marginal error rates, which is the rate one would expect under independence.

The evaluation has three linked parts. First, six model families make 1,152 judgments on 64 executable artifacts from Verus, Lean, SMT, and repository testing. Second, the policies are evaluated on 141 candidates from 47 Python repositories using labels built from tests, downstream checks, and expert review without access to verifier decisions. Third, a generator searches for mutations accepted by each verifier policy.

We find:

1. Cross-model errors are strongly dependent. A held-out trio has 17.0% observed all-member failure versus 3.0% under independence, with joint failures in all four domains.
2. Model-only portfolios reduce some wrong acceptances but reject many more correct candidates. The strongest single model has the highest model-only accuracy in the repository sample.
3. Routing the same model disagreements to pytest instead of a third model gives the strongest observed operating point: 8.5% wrong acceptance, 78.9% correct acceptance, and 84.6% accuracy at 1.91 normalized cost units.
4. Repeating one verifier is vulnerable to targeted search. Three repeated Sonnet judgments accept 11 wrong candidates among 20 accepted mutations, 27.0 points above a nonadaptive reference.

The main contribution is not a permanent ranking of providers. It is an evaluation framework and an empirical result: verifier count overstates independent evidence, and a different verification mechanism can be more valuable than another correlated vote.

2 Related Work

Models as judges and verifiers. LLM-as-a-Verifier studies verification as an inference-time scaling method [20]. FormalRewardBench separates proof generation from proof evaluation [22]. Work on LLM judges documents position, style, and self-preference effects [30]. Repeated sampling can improve generation when reasoning paths contain complementary information [27]. We study whether repeated judgments provide independent evidence of correctness.

Ensembles and correlated error. Classical ensemble work distinguishes member accuracy from error diversity [14, 19]. Recent work on compositional reliability also argues against assuming independent component failures [11]. We provide paired measurements on executable artifacts, repository changes, and candidates optimized against a verifier.

Executable software evaluation. VeriContest, ITPEval, and FVSpec provide formal or executable tasks for evaluating generated artifacts [29, 28, 15]. SWE-bench and later validation work demonstrate both the value and difficulty of repository-level evaluation [18, 12, 21]. Because tests can be incomplete, our repository labels combine execution evidence with semantic review.

3 Evaluation

3.1 Controlled executable study

The controlled study contains 64 candidates from 23 task groups. VeriContest contributes 16 Verus candidates, ITPEval contributes 24 Lean artifacts, FVSpec contributes 12 SMT projections, and repository testing contributes 12 patches with pinned pytest outcomes. Variants of one task remain in the same partition. The grouped split contains 17 calibration candidates and 47 held-out candidates from 15 task clusters.

Six model families judge every candidate: Claude Sonnet 5, Claude Opus 5, Qwen3 Coder, DeepSeek V3.2, Devstral 2, and GPT-5.6 Terra. Models were accessed in August 2026. Open-weight endpoints use temperature zero and seeds 17, 29, and 43. Claude endpoints use extended thinking without a temperature parameter. Terra uses the cross-region inference profile `us.openai.gpt-5.6-terra`, with high reasoning effort and no temperature parameter. The labels and checker outputs are hidden, and the verifier prompt is fixed across models.

The initial output cap is 500 tokens. Forty-five Terra cells truncate under that setting; those cells alone are rerun with lower reasoning effort and a larger output cap, and only completed responses enter the analysis. The final matrix contains 1,152 parsed judgments and no parse failures. We call the setting nominally deterministic because identical hosted requests can still vary. The repeated same-model panel uses three Sonnet calls, and the primary cross-model panel uses Sonnet, Qwen, and DeepSeek.

3.2 Independent repository study

We sample 47 repositories from the 1,000 most downloaded PyPI packages. Each project has a pytest suite, supports Python 3.10 or later, runs its tests in under five minutes, and does not require Docker. Each repository contributes three candidates: a maintainer-accepted pull request, a submitted-then-revised pull request, and a targeted mutation to tested behavior, for 141 candidates.

Labels combine the full test suite, downstream dependent-package tests when available for 89 candidates, and two independent expert reviews against the stated change intent. Reviewers do not see verifier outputs or confidence. One reviewer has eight years of software-engineering experience and three years maintaining Python packages; the other has six years of software-engineering experience and two years in formal-verification research. Both are research-team members performing normal research duties without separate compensation.

Reviewers agree directly on 108 of 141 candidates. Twenty-two cases have one decisive label and one ambiguous label, so the decisive label is retained. Eleven direct correct-versus-wrong disagreements are excluded. The analysis therefore contains 130 candidates, 71 correct and 59 wrong, with $\kappa = 0.73$. The institutional determination was that evaluation of public and synthetically mutated software artifacts, without personal or behavioral outcomes, was not human-subjects research and did not require IRB review.

3.3 Policies

All panel choices are made using the 17 controlled calibration candidates and frozen before held-out and repository evaluation. Single Terra and Single Qwen use run identity 17. Three Sonnet uses run identities 17, 29, and 43 and accepts with at least two votes. The cross-model panel uses Sonnet, Qwen, and DeepSeek at run identity 17 and also uses majority vote.

For the selected trio, we enumerate all three-member panels among 18 model-run identities. Each panel receives

$$0.4A_{\text{maj}} + 0.3(1 - \text{JF}) + 0.2\bar{A} + 0.1F,$$

where A_{maj} is majority accuracy, $\bar{A}$ is mean member accuracy, and F is the number of represented model families. This selects DeepSeek run 43, Devstral run 17, and Opus run 29. The disagreement-selected policy accepts with two votes. The covariance-aware variant additionally requires every member’s confidence to be at least 0.6.

The routing policies first call Qwen and Sonnet. If they agree, their shared verdict is used. On the same 34 disagreements, model-plus-pytest routing uses pytest pass or fail as the decision, while the model-only control uses DeepSeek’s verdict. This is a same-trigger comparison, not a cost-matched comparison. Verification cost is normalized to the median input-plus-output token count of one Sonnet call. A repository pytest escalation costs 0.36 units, giving expected costs $1.82 + 0.262(0.36) = 1.91$ and $1.82 + 0.262(0.78) = 2.02$ for pytest and model-only routing.

3.4 Measurements

Let $e_i(x) = 1$ when verifier i is wrong on candidate x . For panel S , observed all-member failure is

$$\text{JF}(S) = \frac{1}{n} \sum_{x=1}^n \prod_{i \in S} e_i(x). \quad (1)$$

The independence comparison is

$$\text{JF}_{\text{ind}}(S) = \prod_{i \in S} \left(\frac{1}{n} \sum_{x=1}^n e_i(x) \right). \quad (2)$$

For repository policies, we report wrong acceptance, correct acceptance, accuracy, and expected cost. Confidence intervals use 10,000 percentile bootstrap replicates, resampling task clusters in the controlled study and repositories in the repository study. The routing comparison is reported descriptively from its confusion matrices because candidate-level paired bootstrap inputs are not part of the current anonymous package.

4 Results

4.1 Cross-model errors remain strongly dependent

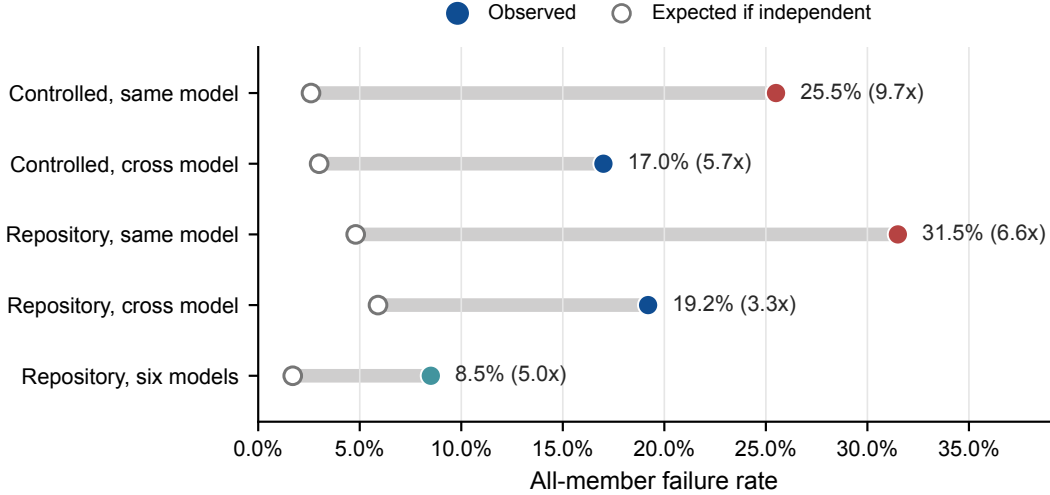


Figure 1: Observed all-member failure and the product of member error rates. Labels show observed failure and its ratio to the independence estimate.

Figure 1 shows the central dependence result. In the controlled held-out set, all three cross-model verifiers are wrong on 8 of 47 candidates, or 17.0%, versus 3.0% under independence. Repeating Sonnet produces 25.5% joint failure versus 2.6%. Cross-model joint failures occur in every domain: 2/10 VeriContest, 2/8 FVSpec, 2/21 ITPEval, and 2/8 repository candidates. The small per-domain counts limit comparisons, but no one source explains the pooled result.

The pattern also appears for alternative pairs on the same held-out candidates. Observed joint failure is 21.3% for DeepSeek and Terra, 19.1% for Sonnet and Opus, and 23.4% for Qwen and DeepSeek, between 3.7 and 6.0 times their independence products.

Repository labels yield the same qualitative conclusion. Repeated same-model verification fails jointly on 31.5%, compared with 4.8% under independence. Cross-model verification lowers observed failure to 19.2%, but its independence estimate is 5.9%. A six-model panel lowers observed failure to 8.5%, still five times its 1.7% independence estimate. Diversity reduces shared failure, but member count remains a poor proxy for independent evidence.

4.2 A different checker changes the operating point

Table 1: Repository results on 130 labeled candidates. Higher correct acceptance and accuracy are better; lower wrong acceptance and cost are better. Costs are normalized verification units.

Policy	Correct acc.	Wrong acc.	Accuracy	Cost
Single Terra	.704	.169	.762	1.08
Single Qwen	.690	.203	.738	.82
Three Sonnet	.451	.136	.638	3.00
Cross-model	.507	.119	.677	2.60
Selected trio	.563	.102	.715	2.84
Covariance-aware	.521	.119	.685	2.84
Same-trigger model	.451	.136	.638	2.02
Model + pytest	.789	.085	.846	1.91

Among model-only policies, Single Terra has the highest accuracy, while the selected trio lowers wrong acceptance from 16.9% to 10.2% but also lowers correct acceptance from 70.4% to 56.3%. The benefit of model diversity is therefore application-dependent.

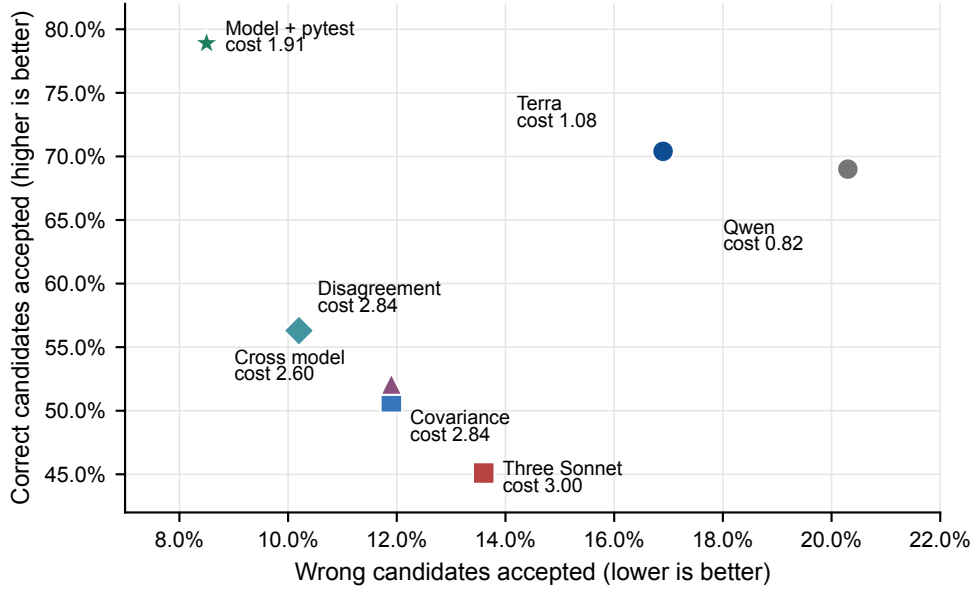


Figure 2: Repository acceptance tradeoff. Moving left accepts fewer wrong candidates; moving up retains more correct candidates. The same-trigger model-only point overlaps Three Sonnet and is omitted for readability.

Routing model disagreements to a different mechanism changes both axes. Model-plus-pytest routing correctly rejects 54 of 59 wrong candidates and accepts 56 of 71 correct candidates, yielding 8.5% wrong acceptance, 78.9% correct acceptance, and 84.6% accuracy. Both routing policies escalate the same 34 cases. With DeepSeek as the escalation, the full-sample policy gives 13.6%, 45.1%, and 63.8%, respectively. The pytest route is also slightly cheaper under our normalization, 1.91 versus 2.02 units.

This result needs one important qualification. Pytest contributes to the composite repository label, together with downstream tests and expert review, so it is not an independent semantic oracle. On escalated cases, pytest agrees with the final composite label on 28 of 34 candidates. The result supports verification-signal consistency and practical routing in this sample, not a claim that one test suite establishes complete correctness.

4.3 Repeated judgments expose a targetable blind spot

Claude Sonnet 4.7 generates return substitutions, boundary shifts, branch flips, and off-by-one mutations. Each policy has a 500-attempt cap and stops after 20 accepted candidates or, once at least 15 are accepted, after 100 consecutive rejections. Exact duplicate source files are removed by SHA-256. An evaluator blinded to the accepting policy uses modified pytest with hidden regression tests, downstream tests when available, and one expert semantic review. Any execution failure or semantic error marks the candidate wrong. On 72 overlapping cases, execution and semantic review agree on 65, or 90.3%.

The nonadaptive reference accepts 75 of 500 candidates, of which 21 are wrong (28.0%, 95% CI [18.2, 39.3]). Three repeated Sonnet judgments accept 11 wrong candidates among 20 accepted (55.0%), a 27.0-point increase with a 95% interval of [6.8, 48.2] points. Intervals for Single Sonnet, the cross-model panel, and the selected trio include zero.

The selected trio accepts 18 candidates during attempts 1–56 and then rejects attempts 57–156, triggering the patience rule. This says mutation yield declined sharply for the tested generator and search procedure. It does not establish that the policy has no remaining blind spots.

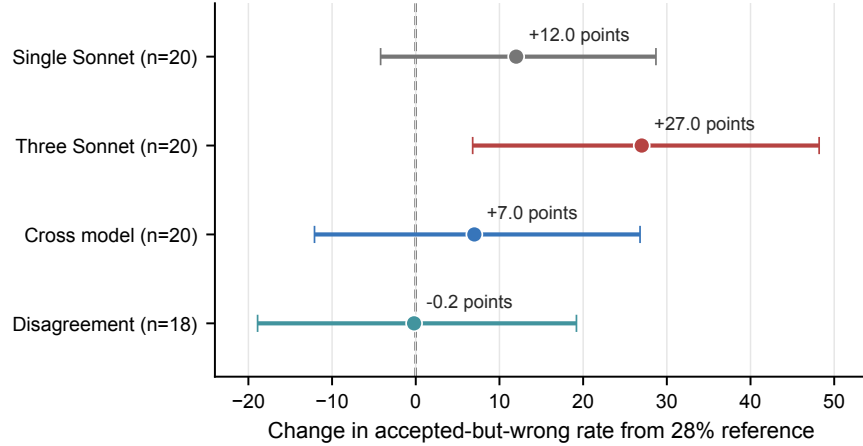


Figure 3: Change in accepted-but-wrong rate under targeted mutation search, relative to the 28% nonadaptive reference. Horizontal lines are 95% confidence intervals.

5 Observed Failure Patterns

Our qualitative review is descriptive, not a separately powered causal analysis. Three recurring patterns help interpret the aggregate results. First, correct Lean and Verus artifacts are sometimes rejected because valid constructs look unfamiliar to a general code model. Second, repository patches can receive convincing verbal approval while violating behavior exposed by full or downstream tests. Third, models often share an interpretation of the intended contract even when their explanations differ. The last pattern is consistent with why cross-model diversity helps without approaching independence.

6 Deployment Implications

The results suggest a concrete workflow. Evaluate candidate verifiers on the same labeled examples, measure joint rather than marginal error, and select an operating point using the relative cost of wrong acceptance and correct rejection. Use disagreement as a routing signal, but route to execution, formal checking, or expert review rather than assuming another model provides independent evidence. Re-evaluate after changing prompts, providers, or model versions.

7 Limitations and Broader Impact

The repository sample excludes projects requiring Docker, long-running tests, older Python environments, or specialized infrastructure. Each repository contributes only three candidates, and the sample mixes natural pull-request states with targeted mutations. These rates are evaluation measurements, not production defect rates.

Tests can miss semantic regressions, downstream checks exist for only 89 candidates, and experts can disagree. We report agreement and exclude direct correct-versus-wrong disagreements. The routing result partially reuses pytest in the label, and its descriptive comparison lacks the candidate-level paired data needed for a clustered uncertainty estimate in the current package.

The adaptive study uses one generator, four mutation operators, and small accepted samples. The short stability check repeats 48 cells over 11 days with unchanged model identifiers and byte-identical prompts. Forty-six decisions remain stable; two flips concern one contested ITPEval theorem, mean absolute confidence change is .022, and aggregate accuracy is unchanged. This measures short-term serving stability, not model-version drift.

The work can help prevent false confidence in model agreement and encourage independent checking. The same measurements can guide searches for verifier blind spots. We report aggregate attack behavior and recommend independent validation before high-consequence use.

8 Conclusion

Several model judgments do not automatically provide several independent checks. Across controlled and repository studies, observed all-member failure is 3.3 to 9.7 times the product of marginal error rates. More model families reduce shared failure but do not remove it. Model-only portfolios also impose substantial rejection costs.

The strongest observed repository policy routes model disagreements to pytest, not another model. Together with the targeted-search result, this supports a simple principle: use model diversity to find uncertainty, then resolve it with evidence produced by a different mechanism.

References

- [1] Adarsh Agrawal and Rahul Suresh Babu. Incident memory: Training-free operational memory through sequential pattern mining and velocity-stratified retrieval. *arXiv preprint arXiv:2609.01616*, 2026. URL <https://arxiv.org/abs/2609.01616>.
- [2] Adarsh Agrawal and Nikhil Garg. Time series forecasting of producer price index, using ARIMA. *International Journal of Science and Research*, 7(7):1019–1024, 2018.
- [3] Adarsh Agrawal and Shashank Indukuri. Schema-first retrieval: Embedding catalogs for natural language analytics. *arXiv preprint arXiv:2606.28387*, 2026. URL <https://arxiv.org/abs/2606.28387>.
- [4] Adarsh Agrawal and J. Li. Mitigating bias in AI using Debias-GAN. In *NVIDIA GTC Conference*, 2022.
- [5] Amazon AGI Intelligence. The amazon nova family of models: Technical report and model card. Technical report, Amazon, 2024. Available from Amazon Science.
- [6] Amazon AGI Intelligence. Amazon nova multimodal embeddings: Technical report and model card. Technical report, Amazon, 2025. Available from Amazon Science.
- [7] Amazon AGI Intelligence. Amazon nova premier: Technical report and model card. Technical report, Amazon, 2025. Available from Amazon Science.
- [8] Amazon AGI Intelligence. Amazon nova sonic: Technical report and model card. Technical report, Amazon, 2025. Available from Amazon Science.
- [9] Amazon AGI Intelligence. Amazon nova 2: Multimodal reasoning and generation models. Technical report, Amazon, 2025. Technical report and model card.
- [10] Rahul Suresh Babu and Adarsh Agrawal. Self-healing agentic orchestrators for reliable tool-augmented large language model systems. *arXiv preprint arXiv:2606.01416*, 2026. URL <https://arxiv.org/abs/2606.01416>.
- [11] V. P. Bhardwaj, G. Singh, and A. P. Bhardwaj. Agent behavioral contracts ii: Certifying compositional reliability without assuming independence. *arXiv preprint arXiv:2608.12895*, 2026.
- [12] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified, 2024. OpenAI benchmark report.
- [13] Nicolas de Condorcet. *Essay on the Application of Analysis to the Probability of Majority Decisions*. 1785.
- [14] Thomas G. Dietterich. Ensemble methods in machine learning. In *Multiple Classifier Systems*, pages 1–15, 2000.
- [15] Quinn Dougherty, Max von Hippel, Simon Henniger, Hazel Shackleton, and Mike Dodds. FVSpec: Real-world property-based tests as lean challenges. *arXiv preprint arXiv:2606.01008*, 2026.

- [16] Wei Feng, Adarsh Agrawal, Haifeng Ling, Erik Blasch, E. Adiles-Cruz, P. T. Schrader, and J. Wei. DDDAS probability learning for natural disaster change detection. In *International Conference on Dynamic Data Driven Applications Systems*, pages 90–99, 2024.
- [17] Shashank Indukuri and Adarsh Agrawal. Grounded optimization: A layered engineering framework for reducing LLM hallucination in automated personal document rewriting. *arXiv preprint arXiv:2607.01457*, 2026. URL <https://arxiv.org/abs/2607.01457>.
- [18] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- [19] Ludmila I. Kuncheva and Christopher J. Whitaker. Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine Learning*, 51(2):181–207, 2003.
- [20] Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. LLM-as-a-verifier: A general-purpose verification framework. *arXiv preprint arXiv:2607.05391*, 2026.
- [21] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities, 2026. Benchmark audit.
- [22] Zeynel A. Uluşan, Burak S. Akbudak, Can S. Erer, and Gözde Gül Şahin. FormalRewardBench: A benchmark for formal theorem proving reward models. *arXiv preprint arXiv:2605.10141*, 2026.
- [23] Bertie Vidgen, Adarsh Agrawal, Ahmed M. Ahmed, Victor Akinwande, Noura Al-Nuaimi, Nada Alfaraj, et al. How to construct AI safety benchmarks, 2024. Companion publication.
- [24] Bertie Vidgen, Adarsh Agrawal, Ahmed M. Ahmed, Victor Akinwande, Noura Al-Nuaimi, Nada Alfaraj, et al. Others. 2024. introducing v0.5 of the AI safety benchmark from MLCommons. *arXiv preprint arXiv:2404.12241*, 2024. URL <https://arxiv.org/abs/2404.12241>.
- [25] Bertie Vidgen, Adarsh Agrawal, Ahmed M. Ahmed, Victor Akinwande, Noura Al-Nuaimi, Nada Alfaraj, et al. Introducing v0.5 of the AI safety benchmark from MLCommons. *arXiv preprint arXiv:2404.12241*, 2024. URL <https://arxiv.org/abs/2404.12241>.
- [26] Bertie Vidgen, Adarsh Agrawal, Ahmed M. Ahmed, Victor Akinwande, Noura Al-Nuaimi, Nada Alfaraj, et al. Introducing v0.5 of the AI safety benchmark from MLCommons, 2024. <https://arxiv.org/abs/2404.12241>, 2024.
- [27] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.
- [28] Jiayi Wu, Robert Joseph George, and Anima Anandkumar. ITPEval: Benchmarking formal translation across interactive theorem provers. *arXiv preprint arXiv:2607.19407*, 2026.
- [29] Zichen Xie, Mrigank Pawagi, Yuxin Liu, Aaditi Rai, Lize Shao, Jr. Berberian, John, Sicong Che, and Wenxi Wang. VeriContest: A competitive-programming benchmark for verifiable code generation. *arXiv preprint arXiv:2605.08553*, 2026.
- [30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.

A Joint-Failure Results

B Repository Policy Details

Table 2: All-member failure. Intervals use task-cluster bootstrap for the controlled study and repository-cluster bootstrap for the repository study.

Study	Panel	Observed	Independent	Ratio	95% CI
Controlled	Same-model	.255	.026	9.7	[.142,.316]
Controlled	Cross-model	.170	.030	5.7	[.071,.213]
Repository	Same-model	.315	.048	6.6	[.182,.419]
Repository	Cross-model	.192	.059	3.3	[.114,.287]
Repository	Six-model	.085	.017	5.0	[.031,.154]

Table 3: Controlled held-out cross-model failures by source. Counts are small, so the domain rows are descriptive.

Source	n	Observed	Independent	95% CI
VeriContest	10	.200	.027	[.050,.400]
FVSpec	8	.250	.042	[.063,.500]
ITPEval	21	.095	.024	[.024,.238]
Repository	8	.250	.031	[.063,.500]

Table 4: Alternative model pairs on the same 47 controlled held-out candidates.

Pair	Observed	Independent	Ratio	95% CI
DeepSeek + Terra	.213	.045	4.7	[.106,.340]
Sonnet + Opus	.191	.051	3.7	[.085,.319]
Qwen + DeepSeek	.234	.039	6.0	[.128,.362]

Table 5: Repository confusion matrices. TP means a wrong candidate is rejected; TN means a correct candidate is accepted.

Policy	TP	FP	TN	FN
Single Terra	49	21	50	10
Single Qwen	47	22	49	12
Three Sonnet	51	39	32	8
Cross-model	52	35	36	7
Selected trio	53	31	40	6
Covariance-aware	52	34	37	7
Same-trigger model	51	39	32	8
Model + pytest	54	15	56	5

Using Terra as the reference, its accuracy advantage over Three Sonnet is 12.4 points (95% CI [3.2, 22.1], $p = 0.009$). Accuracy differences against cross-model voting, the selected trio, and the covariance-aware panel have intervals that include zero: $[-1.7, 19.2]$, $[-5.8, 15.9]$, and $[-2.4, 18.6]$, respectively.

C Adaptive Evaluation Details

Table 6: Targeted mutation search. The target is 20 acceptances, with a 500-attempt cap and the patience rule described in Section 4.3.

Policy	Attempts	Accepted	Wrong	Wrong rate
Nonadaptive reference	500	75	21	.280
Single Sonnet	487	20	8	.400
Three Sonnet	312	20	11	.550
Cross-model	201	20	7	.350
Selected trio	156	18	5	.278

Changes from the 28.0% reference are 12.0 points for Single Sonnet (95% CI $[-4.2, 28.7]$), 27.0 for Three Sonnet ($[6.8, 48.2]$), 7.0 for cross-model voting ($[-12.1, 26.8]$), and -0.2 for the selected trio ($[-18.9, 19.2]$).

D Reproducibility, Compute, and Assets

Inference uses hosted endpoints with at most 10 parallel requests per model. No model is trained or fine-tuned, and no local GPU is used. Execution and analysis run on Ubuntu 24.04 with 16 vCPUs and 64 GB RAM. Wall-clock time is 6.4 hours for the controlled study, 3.2 hours for repository evaluation, 14.8 hours for adaptive evaluation, 2.1 hours for native checking, and 0.3 hours for analysis, totaling 26.8 hours. Provisioned compute is 428.8 vCPU-hours; this is capacity-time, not measured CPU utilization.

Tool versions are Verus 0.2689, Lean 4.33.0, Z3 4.15.4 with cvc5 1.3.4, and pytest 8.3.1 with Python 3.12.7.

The planned anonymous supplement will contain candidate-group metadata, verifier and policy decisions, aggregate result tables, and scripts that regenerate reported metrics and figures. The current working package contains the aggregate result data and deterministic figure code. Candidate-level data and the end-to-end analysis script are still being finalized.

Table 7: Licenses for evaluation assets. Repository licenses were checked before inclusion.

Asset	License
VeriContest	CC-BY-4.0
ITPEval / LeanDojo v1.0.2	MIT
FVSpec	Apache-2.0
packaging	Apache-2.0 / BSD-2-Clause
pluggy, pytest-mock, cattrs	MIT
Click, ItsDangerous	BSD-3-Clause
filelock	Public domain / Unlicense